Bellcorp Studio — ATM Simulation Application

System Design Document • ACID Concurrency Control & Engineering Specification

Application: ATM Simulation Application	Stack: React 18 • Node/Express (TS) • PostgreSQL 15 • Redis 7 • MongoDB 6.0
Assignment: Bellcorp Studio Batch 08	Verification: 9/9 Automated Tests PASS • Docker Services Verified • Concurrency Verified

1. Problem Understanding & Concurrency Challenge

An Automated Teller Machine (ATM) executes financial transactions where account balances and physical vault inventory must remain strictly consistent and never become negative under concurrent traffic. The system supports complete banking workflows: secure PIN authentication, live balance inquiries, cash withdrawals, immutable transaction history lookup, and vault cash monitoring.

Concurrency Race Condition:

Consider an account holding ₹3,000.00 that receives two simultaneous withdrawal requests of ₹2,000.00 (Request A and Request B) within the same millisecond:

- Without Concurrency Control (Naive Architecture): Both requests read ₹3,000, both pass balance threshold checks (₹3,000 ≥ ₹2,000), and both commit deductions. The final balance drops to ₹-1,000.00, causing an illegal overdraft and direct financial loss.
- Our ACID Solution (Pessimistic Row-Level Locking): Native PostgreSQL SELECT ... FOR UPDATE locks serialize execution at the database engine level. Request A acquires the lock, validates funds, deducts ₹2,000.00, and commits (leaving ₹1,000.00). Request B unblocks, reads the committed ₹1,000.00, fails validation (₹1,000 < ₹2,000), commits a permanent FAILED record in the ledger with reason INSUFFICIENT_BALANCE, and returns HTTP 409 Conflict. Final balance is strictly ₹1,000.00.

2. High-Level Architecture & Storage Separation

The application employs a decoupled, multi-tiered architecture strictly separating transactional consistency from auxiliary concerns:

Tier / Technology	Role & Architectural Responsibility	Consistency & Failure Behavior
Frontend Tier React 18 + Vite + TypeScript	Responsive banking terminal dashboard, quick denomination pills, live transaction ledger, and real-time Concurrency Safety Lab.	Stateless client consuming REST APIs with JWT Bearer authentication.
Backend API Node.js + Express (TS)	RESTful API gateway, Zod input validation, JWT verification, rate limiting, and ACID transaction orchestrator.	Stateless Node.js processes easily scaled horizontally behind load balancers.
Primary DB PostgreSQL 15 (Docker)	Authoritative Financial Source of Truth: Manages accounts, atm vault, and withdrawals ledger.	Strict ACID compliance, native SELECT FOR UPDATE row locks, and CHECK (balance >= 0) constraints.
Cache Layer Redis 7 (Docker)	High-speed read cache for balance queries (atm:balance:<id>, 60s TTL) and sliding-window rate limiter (10 req/min).	Cache-aside with instant DEL invalidation on commit. Transparent fallback to PostgreSQL on Redis outage.
Audit Store MongoDB 6.0 (Docker)	Audit event store capturing asynchronous activity and compliance events (WITHDRAWAL_SUCCESS, WITHDRAWAL_FAILED, etc.).	Asynchronous, non-blocking post-commit emission. MongoDB failure never rolls back committed financial transactions.

3. Two-Account Architecture & State Isolation

To guarantee that real database concurrency demonstrations never corrupt normal user banking sessions, the database is seeded with a deterministic two-account model:

Property	Account #1 (Demo User)	Account #2 (Concurrency Sandbox)
Account Number	`10000001`	`10000002`
Holder Name	Demo User	Concurrency Sandbox
Baseline Balance	₹10,000.00	₹3,000.00
Primary Purpose	Standard banking operations (login, ₹1,000 withdrawals, balance checks, transaction history)	Dedicated sandbox for live concurrent stress testing (2x ₹2,000 simultaneous calls)
User Access	User-facing PIN authentication (1234)	Internal test account only; no user login flow
Runtime Isolation	Balance updates dynamically on user actions (e.g. ₹10,000 → ₹9,000)	Concurrency test executes real row locks against Account #2 without affecting Account #1

Real Database Execution Guarantee: The Concurrency Safety Lab dispatches real parallel HTTP requests executing WithdrawalService.withdraw with native PostgreSQL SELECT ... FOR UPDATE row locks. It is not an animation or simulated mock.

4. Database Schema & Index Design

PostgreSQL 15 serves as the single transactional source of truth. Integrity is enforced via foreign keys and database constraints.

```
-- 1. Accounts Table
CREATE TABLE accounts (
    id SERIAL PRIMARY KEY,
    account_number VARCHAR(20) UNIQUE NOT NULL,
    holder_name VARCHAR(100) NOT NULL,
    pin_hash VARCHAR(255) NOT NULL,
    balance NUMERIC(12, 2) NOT NULL DEFAULT 0.00 CHECK (balance >= 0),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. ATM Vault Inventory Table
CREATE TABLE atm (
    id SERIAL PRIMARY KEY,
    available_cash NUMERIC(12, 2) NOT NULL DEFAULT 0.00 CHECK (available_cash >= 0),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Withdrawals Ledger Table (Records all SUCCESS and FAILED transactions)
CREATE TABLE withdrawals (
    id SERIAL PRIMARY KEY,
    account_id INTEGER NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
    atm_id INTEGER NOT NULL REFERENCES atm(id) ON DELETE CASCADE,
    amount NUMERIC(12, 2) NOT NULL CHECK (amount > 0),
    status VARCHAR(20) NOT NULL, -- 'SUCCESS' or 'FAILED'
    failure_reason VARCHAR(255),
    balance_before NUMERIC(12, 2),
    balance_after NUMERIC(12, 2),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Performance Indexes
CREATE INDEX idx_accounts_account_number ON accounts(account_number);
CREATE INDEX idx_withdrawals_account_created ON withdrawals(account_id, created_at DESC);
CREATE INDEX idx_withdrawals_status ON withdrawals(status);
```

Composite Index Optimization: idx_withdrawals_account_created enables high-speed bounded index scans for WHERE account_id = $1 ORDER BY created_at DESC LIMIT $2 without in-memory sorting.

5. API Contract & Endpoint Specification

Method & Route	Auth / Guard	Purpose & Request Payload	Response & Status
`POST /api/auth/login`	Public	Authenticate user PIN. { accountNumber, pin }	200 OK: JWT token & account profile. 401: Invalid credentials.
`GET /api/account/balance`	JWT Bearer	Fetch live balance via Redis cache-aside (60s TTL).	200 OK: { balance, isCached }
`POST /api/withdraw`	JWT + Limiter	Execute row-locked withdrawal. { amount, atmId? }	200 OK: New balance. 409: INSUFFICIENT_BALANCE.
`GET /api/transactions`	JWT Bearer	Retrieve bounded history. Query: ?limit=20	200 OK: List of SUCCESS & FAILED records.
`GET /api/atm/status`	Public	Query physical ATM vault available cash.	200 OK: { availableCash, id }
`POST /api/dev/reset-seed`	devGuard	Restore Account #1 (₹10k), #2 (₹3k), ATM (₹50k).	200 OK: (Returns 404 in production).
`POST /api/dev/concurrency-test`	devGuard	Execute 2x ₹2,000 parallel calls on Account #2.	200 OK: Execution metrics report.
`GET /health`	Public	Server uptime and container health status check.	200 OK: { status: 'ok' }

6. Withdrawal Transaction Flow & Deterministic Locking

To prevent deadlocks and eliminate race conditions across multiple ATMs and concurrent digital channels, all withdrawal requests follow a strict, deterministic lock acquisition lifecycle:

Step 1 (BEGIN): Start PostgreSQL ACID transaction.

Step 2 (Pessimistic Account Lock): SELECT balance FROM accounts WHERE id = $1 FOR UPDATE; — Locks the target account row exclusively. Concurrent requests for this account wait.

Step 3 (Pessimistic ATM Vault Lock): SELECT available_cash FROM atm WHERE id = $2 FOR UPDATE; — Locks the ATM inventory row in identical global sequence to eliminate deadlocks.

Step 4 (Validation & Rejection Branch): If balance < amount or available_cash < amount:
- Execute INSERT INTO withdrawals ... ('FAILED', reason, balance, balance);
- Execute COMMIT; to permanently preserve the failure attempt in the ledger.
- Return HTTP 409 Conflict (Account balance and ATM cash remain untouched).

Step 5 (Mutation & Success Branch): If funds and cash are sufficient:
- UPDATE accounts SET balance = balance - $amount WHERE id = $1;
- UPDATE atm SET available_cash = available_cash - $amount WHERE id = $2;
- INSERT INTO withdrawals ... ('SUCCESS', NULL, balanceBefore, balanceAfter);
- Execute COMMIT; (Releases all PostgreSQL row locks).

Step 6 (Post-Commit Cache Invalidation): Call Redis DEL atm:balance:$accountId so subsequent reads fetch fresh PostgreSQL data.

Step 7 (Asynchronous Audit Emit): Emit non-blocking audit event to MongoDB. Return HTTP 200 OK to client.

7. Concurrency Protection & Race Condition Analysis

The following timeline traces two simultaneous ₹2,000.00 withdrawal requests on Sandbox Account #2 (Initial balance ₹3,000.00):

Timeline	Request A (₹2,000.00)	Request B (₹2,000.00)	PostgreSQL Balance
T0	Arrives at API Gateway	Arrives simultaneously at API Gateway	₹3,000.00 (Initial)
T1	Acquires SELECT ... FOR UPDATE lock on Account #2	Blocked waiting for Account #2 row lock	₹3,000.00 (Locked by Req A)
T2	Validates ₹3,000 ≥ ₹2,000 (Passes). Deducts ₹2,000.	Waiting...	₹1,000.00 (Uncommitted)
T3	Inserts SUCCESS row & executes COMMIT. Returns HTTP 200.	Unblocks & acquires lock. Reads committed balance (₹1,000).	**₹1,000.00 (Committed)**
T4	Complete.	Validates ₹1,000 < ₹2,000 (Fails). Inserts FAILED row & commits. Returns HTTP 409.	**₹1,000.00 (Strictly Preserved)**

8. Redis Caching & Rate Limiting Strategy

Cache-Aside Flow: Balance inquiries check Redis key atm:balance:<id> first. Cache hits return directly from Redis without querying PostgreSQL. Cache misses query PostgreSQL and populate Redis with a 60-second TTL. Upon successful cash withdrawal, Redis cache is invalidated immediately via DEL. A sliding-window rate limiter restricts POST /api/withdraw to 10 requests/minute. Fault Behavior: If Redis is offline, balance reads transparently query PostgreSQL directly, and the rate limiter fails open, ensuring banking cash availability is never blocked by cache outages.

9. MongoDB Asynchronous Audit Strategy

Audit Trail Decoupling: Compliance events (WITHDRAWAL_SUCCESS, WITHDRAWAL_FAILED, BALANCE_CHECK, LOGIN) are logged to MongoDB collection activitylogs. Failure Isolation Guarantee: MongoDB writes execute asynchronously post-commit. MongoDB downtime or network latency never blocks, corrupts, or rolls back committed PostgreSQL transactions.

10. Security & Defensive Engineering

- Bcrypt PIN Hashing: Account PINs are salted and hashed (10 rounds). Plaintext PINs are never persisted, logged, or exposed.
- Stateless JWT Tokens: Authenticated sessions utilize signed HMAC-SHA256 JSON Web Tokens with 24-hour expiration.
- SQL Injection Defense: All PostgreSQL interactions utilize strict parameterized queries ($1, $2).
- Development Endpoint Gating: POST /api/dev/reset-seed and POST /api/dev/concurrency-test are guarded by devGuard middleware and return HTTP 404 in production mode.

11. Transaction History & Reset Demo State Semantics

Immutable Financial Ledger: The PostgreSQL withdrawals table maintains a permanent historical record of every transaction attempt. Successful withdrawals record the balance transition (e.g. ₹10,000 → ₹9,000), while failed withdrawals record the failure reason (e.g. INSUFFICIENT_BALANCE) with balance unchanged. The frontend renders failed transactions with a distinct rose-red badge.

Reset Demo State Semantics: Calling POST /api/dev/reset-seed restores live operational balances (Account #1 to ₹10,000.00, Account #2 to ₹3,000.00, ATM Vault to ₹50,000.00) and clears Redis balance caches. Crucially, Reset Demo State NEVER deletes historical transaction ledger records or MongoDB audit logs, preserving financial compliance audit integrity. The UI automatically resets form inputs, deselects denominations, and clears temporary alerts.

12. 100x Horizontal Scalability Strategy (Architectural Roadmap)

Note: This section outlines the architectural scale-out strategy for 100x traffic (thousands of concurrent transactions/sec) and is distinguished from the verified single-node Docker setup.

Scaling Dimension	Current Baseline (Docker Dev)	100x Production Scale-Out Strategy
Application Tier	Single Node.js/Express instance.	Stateless Node.js cluster deployed across autoscaling container groups behind an Application Load Balancer (ALB/Nginx).
Connection Pooling	Direct pg.Pool connection management.	PgBouncer Cluster: Multiplexes thousands of client requests over a compact pool of dedicated PostgreSQL connections via Transaction Pooling.
Primary DB Writes	Single PostgreSQL 15 container.	Strict Financial Consistency Rule: All cash withdrawals and balance updates MUST execute on the Primary PostgreSQL instance with SELECT ... FOR UPDATE. Authoritative balance writes are never routed to read replicas.
Read Replicas	Not deployed in dev baseline.	PostgreSQL streaming read replicas offload non-authoritative read traffic (e.g. historical statement exports and analytical reporting).
Ledger Partitioning	Single withdrawals table.	Declarative Range Partitioning: Partition withdrawals table by date (monthly/quarterly) to maintain fast index scans as data grows by millions of rows.
Cache Tier	Single Redis 7 container.	Redis Cluster: Master-replica sharding across availability zones for high-throughput distributed caching and rate limiting.
Audit Pipeline	Direct async Mongoose insert.	Kafka / RabbitMQ Message Queue: Express instances emit events to message queues; worker services batch-consume records into sharded MongoDB.

13. Architectural Trade-offs Matrix

Architectural Decision	Chosen Strategy	Alternative Considered	Rationale & Impact
Concurrency Control	Pessimistic Row Locking (SELECT FOR UPDATE)	Optimistic Locking (version column)	Pessimistic locking eliminates transaction retry storms under high contention and guarantees deterministic ordering for financial withdrawals.
Cache Consistency	Immediate DEL Invalidation on Commit	Write-Through Cache Mutation	Invalidation prevents race conditions where asynchronous write-through cache mutations overwrite fresh data with stale states.
Audit Resilience	Asynchronous Non-Blocking MongoDB	Synchronous Distributed 2PC	Financial withdrawals must never abort or experience latency due to secondary audit store outages.
Rate Limiter Failure	Fail Open	Fail Closed (Block All Withdrawals)	In banking terminals, critical availability of cash access is prioritized over auxiliary rate-limit enforcement during cache node downtime.

14. Testing & Runtime Verification Results

The system has undergone automated testing, container health audits, and live runtime verification against real Docker services:

Verification Category	Test Target & Scenario	Observed Runtime Result	Status
Docker Services	PostgreSQL 15, MongoDB 6.0, Redis 7	All 3 containers healthy on ports 5432, 27017, 6379.	**PASS**
Authentication	POST /api/auth/login (10000001 / 1234)	Bcrypt verified; signed JWT Bearer issued.	**PASS**
Normal Withdrawal	Withdraw ₹1,000 on Account #1 (₹10,000 balance)	Balance becomes ₹9,000; ATM cash drops to ₹49,000; ledger recorded.	**PASS**
Overdraft Rejection	Withdraw ₹11,000 on Account #1 (₹10,000 balance)	HTTP 409 INSUFFICIENT_BALANCE; balance preserved at ₹10,000; FAILED ledger row inserted.	**PASS**
Real Concurrency Test	2x ₹2,000 simultaneous calls on Account #2 (₹3,000)	1 SUCCESS, 1 FAILED; final balance strictly ₹1,000; ATM deducted ₹2,000; Account #1 unaffected.	**PASS**
Account Isolation	Inspect Account #1 after Account #2 Concurrency Test	Account #1 balance strictly preserved (100% isolated).	**PASS**
Reset Demo State	POST /api/dev/reset-seed	Restores Account #1 (₹10k), Account #2 (₹3k), ATM (₹50k); preserves all historical transaction records.	**PASS**
Redis Cache Cycle	1st Read (Miss) → 2nd Read (Hit) → Withdraw (Invalidate)	Redis cache invalidated; next read queries PostgreSQL.	**PASS**
Automated Test Suite	Jest backend test runner (npm test)	9/9 tests passed across atm.test.ts and concurrency.test.ts.	**PASS**
Production Build	Vite client compiler (npm run build)	Compiled successfully in 3.18s with 0 errors.	**PASS**